



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Інженерія програмного забезпечення

Практична робота №5

«Шаблони поведінки. Шаблони Visitor, Memento, State, Command, Interpreter»

Виконав: студент 2 курсу ФІОТ, гр. ІО-41

Давидчук А. М.

Залікова книжка № 4106

Перевірив: *Васильєва М. Д.*

Тема: «Шаблони поведінки. Шаблони Visitor, Memento, State, Command, Interpreter».

Мета: Ознайомлення з видами шаблонів проектування ПЗ. Вивчення шаблонів поведінки. Отримання базових навичок з застосування шаблонів Iterator, Mediator, Observer, Chain of Responsibility.

Практична частина (Завдання №1)

Мій номер залікової книги: 4106, звідси $4106 \bmod 8 = 2$. Тому умова до 1 завдання буде:

Необхідно створити модель мережевої структури, що складається з таких типів елементів:

- **Cable** — елемент, який представляє мережевий кабель;
- **Server** — елемент, який представляє сервер;
- **Workstation** — елемент, який представляє робочу станцію;
- **Network** — комплексний елемент, який може містити інші елементи мережі (сервери, кабелі, робочі станції, а також вкладені підмережі).

Структура повинна підтримувати виконання додаткових операцій (наприклад, розрахунок вартості, відображення конфігурації тощо) **без внесення змін у класи елементів** (Cable, Server, Workstation, Network).

Вимоги

- **Абстрактний клас або інтерфейс елемента мережі (NetworkElement):**
 - описує спільну поведінку всіх елементів мережі;
 - повинен містити метод для прийому обробника (наприклад, `accept(Visitor v)`), який дозволяє елементу “передати себе” об’єкту, що виконує певну операцію.
- **Реалізувати класи елементів мережі:**
 - **Cable**;
 - **Server**;
 - **Workstation**;
 - **Network** (може містити інші елементи мережі, у тому числі вкладені об’єкти Network).
- **Механізм розширення поведінки елементів мережі:**
 - повинен дозволяти виконувати над елементами нові дії (розрахунок вартості, виведення структури, перевірка стану тощо) **без зміни коду класів елементів**;
 - для цього необхідно передбачити узгоджений набір методів обробки для кожного типу елемента.
- **Клас, який реалізує конкретну операцію над елементами мережі:**
 - наприклад, клас для розрахунку вартості мережі або для відображення її конфігурації;

- повинен містити окремі методи для обробки різних типів елементів: `Cable`, `Server`, `Workstation`, `Network`.

- Спеціальний метод прийому обробника в кожному елементі мережі:

- елемент мережі не повинен знати, яку саме дію виконає обробник;
- елемент лише викликає метод обробника, передаючи йому себе як параметр (приклад: `visitor.visitCable(this);`).

У методі `main` продемонструвати:

- створення мережевої структури з декількох елементів (мережа, яка містить сервер, робочу станцію, кабелі, підмережі);
- виконання різних операцій над елементами мережі;
- можливість додавання нових типів операцій (наприклад, `Display` для відображення конфігурації або `CheckConnection` для перевірки зв'язності між вузлами) без зміни коду класів `Network`, `Server`, `Workstation`, `Cable`.

Загальна структура проекту до компіляції:

```
1 PS C:\Users\artem\Desktop\reps\university_roadmap\semestr3\software
   ↪ engineering\labs\lab5\task1> tree /f /a
2 Folder PATH listing
3 Volume serial number is 1A5B-F8AF
4 C:.
5 |   build.xml
6 |
7 |---src
8 |   |---work5
9 |       Cable.java
10 |       CostCalculatorVisitor.java
11 |       DisplayVisitor.java
12 |       Network.java
13 |       NetworkDemo.java
14 |       NetworkElement.java
15 |       NetworkVisitor.java
16 |       Server.java
17 |       Workstation.java
```

Код `build.xml` задля автоматизації:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project name="task1" default="run" basedir=".">
3
4     <!-- Основні директорії -->
5     <property name="src.dir" value="src"/>
6     <property name="build.dir" value="build"/>
7     <property name="dist.dir" value="dist"/>
8     <property name="doc.dir" value="doc"/>
9
10    <!-- Ініціалізація: створення службових папок -->
```

```

11 <target name="init" description="Create build, dist and doc directories">
12     <mkdir dir="${build.dir}"/>
13     <mkdir dir="${dist.dir}"/>
14     <mkdir dir="${doc.dir}"/>
15 </target>
16
17 <!-- Клас-пас для компіляції та запуску -->
18 <path id="classpath">
19     <pathelement path="${build.dir}"/>
20 </path>
21
22 <!-- Компіляція Java-джерел -->
23 <target name="compile" depends="init" description="Compile Java sources">
24     <javac srcdir="${src.dir}"
25         destdir="${build.dir}"
26         includeantruntime="false"
27         source="1.8"
28         target="1.8"
29         debug="true"
30         encoding="UTF-8">
31         <classpath refid="classpath"/>
32     </javac>
33 </target>
34
35 <!-- Створити виконуваний JAR-файл -->
36 <target name="jar" depends="compile" description="Create runnable JAR archive">
37     <jar destfile="${dist.dir}/task1.jar" basedir="${build.dir}">
38         <manifest>
39             <attribute name="Main-Class" value="work5.NetworkDemo"/>
40         </manifest>
41     </jar>
42 </target>
43
44 <!-- Запуск програми -->
45 <target name="run" depends="compile,javadoc" description="Run demo program">
46     <java classname="work5.NetworkDemo" fork="true">
47         <classpath>
48             <pathelement path="${build.dir}"/>
49         </classpath>
50     </java>
51 </target>
52
53 <!-- Генерація Javadoc-документації для пакету work5 -->
54 <target name="javadoc" depends="compile" description="Generate Javadoc
55 ↪ documentation">
56     <javadoc destdir="${doc.dir}"
57         sourcepath="${src.dir}"
58         packagenames="work5"
59         encoding="UTF-8"
60         docencoding="UTF-8"
61         charset="UTF-8"
62         author="true"
63         version="true"
64         use="true">

```

```

64         windowtitle="task1 API"
65         doctitle="task1 API">
66     </javadoc>
67 </target>
68
69 <!-- Прибирання результатів збірки -->
70 <target name="clean" description="Delete build, dist and doc directories">
71     <delete dir="${build.dir}"/>
72     <delete dir="${dist.dir}"/>
73     <delete dir="${doc.dir}"/>
74 </target>
75
76 </project>

```

Перевірка:

```

1 PS C:\Users\artem\Desktop\reps\university_roadmap\semestr3\software
  ↪ engineering\labs\lab5\task1> ant -p
2 Buildfile: C:\Users\artem\Desktop\reps\university_roadmap\semestr3\software
  ↪ engineering\labs\lab5\task1\build.xml
3
4 Main targets:
5
6 clean      Delete build, dist and doc directories
7 compile    Compile Java sources
8 init       Create build, dist and doc directories
9 jar        Create runnable JAR archive
10 javadoc    Generate Javadoc documentation
11 run        Run demo program
12 Default target: run

```

Вихідний код буде на віддаленому репозиторії, посилання на яке буде додане разом із звітом.

Загальна структура проекту після виконання збірки:

```

1 PS C:\Users\artem\Desktop\reps\university_roadmap\semestr3\software
  ↪ engineering\labs\lab5\task1> tree /f /a
2 Folder PATH listing
3 Volume serial number is 1A5B-F8AF
4 C:..
5 |   build.xml
6 |
7 +---build
8 |   \---work5
9 |       Cable.class
10 |       CostCalculatorVisitor.class
11 |       DisplayVisitor.class
12 |       Network.class
13 |       NetworkDemo.class
14 |       NetworkElement.class
15 |       NetworkVisitor.class
16 |       Server.class
17 |       Workstation.class

```

```

18 |
19 +---dist
20 +---doc
21 | | allclasses-index.html
22 | | allpackages-index.html
23 | | copy.svg
24 | | element-list
25 | | help-doc.html
26 | | index-all.html
27 | | index.html
28 | | link.svg
29 | | member-search-index.js
30 | | module-search-index.js
31 | | overview-tree.html
32 | | package-search-index.js
33 | | script.js
34 | | search-page.js
35 | | search.html
36 | | search.js
37 | | stylesheet.css
38 | | tag-search-index.js
39 | | type-search-index.js
40 |
41 | +---legal
42 | | COPYRIGHT
43 | | jquery.md
44 | | jqueryUI.md
45 | | LICENSE
46 |
47 | +---resources
48 | | glass.png
49 | | x.png
50 |
51 | +---script-dir
52 | | jquery-3.7.1.min.js
53 | | jquery-ui.min.css
54 | | jquery-ui.min.js
55 |
56 | \---work5
57 | | Cable.html
58 | | CostCalculatorVisitor.html
59 | | DisplayVisitor.html
60 | | Network.html
61 | | NetworkDemo.html
62 | | NetworkElement.html
63 | | NetworkVisitor.html
64 | | package-summary.html
65 | | package-tree.html
66 | | package-use.html
67 | | Server.html
68 | | Workstation.html
69 |
70 | \---class-use
71 | | Cable.html

```

```

72 |         CostCalculatorVisitor.html
73 |         DisplayVisitor.html
74 |         Network.html
75 |         NetworkDemo.html
76 |         NetworkElement.html
77 |         NetworkVisitor.html
78 |         Server.html
79 |         Workstation.html
80 |
81 | \---src
82 |     \---work5
83 |         Cable.java
84 |         CostCalculatorVisitor.java
85 |         DisplayVisitor.java
86 |         Network.java
87 |         NetworkDemo.java
88 |         NetworkElement.java
89 |         NetworkVisitor.java
90 |         Server.java
91 |         Workstation.java

```

Скріншот наявності документації:

Package work5

package work5

All Classes and Interfaces

Interfaces

Classes

Class	Description
Cable	Елемент мережі, що представляє мережевий кабель.
CostCalculatorVisitor	Конкретний відвідувач, який обчислює загальну вартість усієї мережі.
DisplayVisitor	Конкретний відвідувач, який виводить структуру мережі у вигляді дерева.
Network	Композитний елемент мережі, що представляє логічну мережу або підмережу.
NetworkDemo	Демонстраційний клас з методом main, що показує використання шаблонів С.
NetworkElement	Базовий інтерфейс елемента мережі.
NetworkVisitor	Інтерфейс відвідувача для елементів мережі.
Server	Елемент мережі, що представляє сервер.

Рис. 1: Скріншот документації Javadoc для 1 завдання

UML-діаграма проекту:

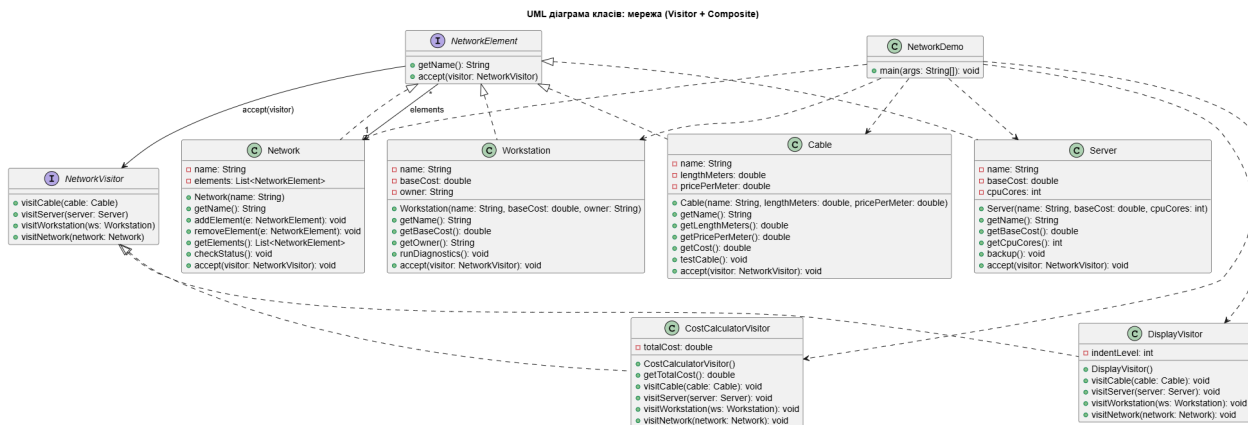


Рис. 2: UML-діаграма проекту для 1 завдання

У пакеті `work5` реалізовано модель мережі згідно з варіантом: класи `Cable`, `Server`, `Workstation` і `Network` імплементують інтерфейс `NetworkElement`.

Клас `Network` містить колекцію `List<NetworkElement>`, тобто мережа може включати сервери, робочі станції, кабелі та підмережі, що відповідає структурі шаблону *Composite*. Для розширення поведінки без зміни цих класів використано шаблон *Visitor*: інтерфейс `NetworkVisitor` та конкретні відвідувачі `DisplayVisitor` (виведення структури) і `CostCalculatorVisitor` (розрахунок загальної вартості). Нові операції можна додавати створенням нових відвідувачів без модифікації кодів елементів, як вимагає умова. У класах визначені бізнес-методи-заглушки (наприклад, `backup()`, `runDiagnostics()`), які лише виводять інформаційні повідомлення. Клас `NetworkDemo` у методі `main()` створює приклад мережі, викликає ці методи та демонструє роботу відвідувачів. Збірка, запуск та генерація Javadoc виконуються за допомогою Ant через файл `build.xml`, що підтверджується успішним `BUILD SUCCESSFUL`.

Практична частина (Завдання №2)

Мій номер залікової книги: 4106, звідси $4106 \bmod 7 = 4$. Тому умова до 2 завдання буде: Необхідно створити модель взаємодії користувача з пунктами меню та кнопками інструментальної панелі, яка включає такі елементи:

- **Action (Дія)** — абстрактний опис реакції на натискання кнопки або вибір пункту меню;
- **ConcreteAction (Конкретна дія)** — конкретні реалізації дій, які визначають, що відбувається при натисканні (наприклад, `SaveFileAction`, `OpenFileAction`, `CopyAction`, `PasteAction`);
- **Invoker (Ініціатор)** — клас, який відповідає за “натискання” кнопок чи вибір пунктів меню і передачу виконання дій відповідним об’єктам `Action`.

Вимоги

- Створити інтерфейс або абстрактний клас `Action`, який описує метод `execute()`.
- Реалізувати декілька класів дій (`SaveFileAction`, `OpenFileAction`, `CopyAction`, `PasteAction`), які наслідують `Action`. Кожен клас визначає, яка поведінка відбувається при взаємодії користувача.

- Клас `Invoker` має містити посилання на поточну дію. Натискання кнопки чи вибір пункту меню викликає метод `execute()` поточної дії. Зміна дії динамічно змінює реакцію `Invoker` без використання умовних операторів.
- Реалізувати можливість формування макро-реакцій — послідовностей дій, що виконуються за одного виклику (клас `MacroAction`, який містить список об'єктів `Action` і при виконанні викликає `execute()` для кожної з них).

У методі `main` продемонструвати

- створення кількох дій (`Save`, `Open`, `Copy`, `Paste`) та об'єкта `Invoker`;
- виконання натискань кнопок і демонстрацію того, що `Invoker` делегує поведінку поточній дії;
- динамічну зміну дії `Invoker` та демонстрацію нової поведінки без використання умовних операторів.

Загальна структура проекту до компіляції:

```

1 PS C:\Users\artem\Desktop\reps\university_roadmap\semestr3\software
   ↪ engineering\labs\lab5\task2> tree /f /a
2 Folder PATH listing
3 Volume serial number is 1A5B-F8AF
4 C:..
5 |   build.xml
6 |
7 |---src
8 |   \---work5
9 |       Action.java
10 |       ActionDemo.java
11 |       CopyAction.java
12 |       Invoker.java
13 |       MacroAction.java
14 |       OpenFileAction.java
15 |       PasteAction.java
16 |       SaveFileAction.java

```

Код `build.xml` задля автоматизації:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <project name="task2" default="run" basedir=".">
3
4     <!-- Основні директорії проекту -->
5     <property name="src.dir" value="src"/>
6     <property name="build.dir" value="build"/>
7     <property name="dist.dir" value="dist"/>
8     <property name="doc.dir" value="doc"/>
9
10    <!-- Початкова ініціалізація: створення службових папок -->
11    <target name="init" description="Create build, dist and doc directories">
12        <mkdir dir="${build.dir}"/>
13        <mkdir dir="${dist.dir}"/>

```

```

14     <mkdir dir="${doc.dir}"/>
15 </target>
16
17 <!-- Клас-пас для компіляції та запуску -->
18 <path id="classpath">
19     <pathelement path="${build.dir}"/>
20 </path>
21
22 <!-- Компіляція вихідних Java-файлів -->
23 <target name="compile" depends="init" description="Compile Java sources">
24     <javac srcdir="${src.dir}"
25         destdir="${build.dir}"
26         includeantruntime="false"
27         source="1.8"
28         target="1.8"
29         debug="true"
30         encoding="UTF-8">
31         <classpath refid="classpath"/>
32     </javac>
33 </target>
34
35 <!-- Генерація Javadoc-документації для пакета work5 -->
36 <target name="javadoc" depends="compile" description="Generate Javadoc
37 ↪ documentation">
38     <javadoc destdir="${doc.dir}"
39         sourcepath="${src.dir}"
40         packagenames="work5"
41         encoding="UTF-8"
42         docencoding="UTF-8"
43         charset="UTF-8"
44         author="true"
45         version="true"
46         use="true"
47         windowtitle="task2 API"
48         doctitle="task2 API">
49     </javadoc>
50 </target>
51
52 <!-- Створення виконуваного JAR-архіву -->
53 <target name="jar" depends="compile" description="Create runnable JAR archive">
54     <jar destfile="${dist.dir}/task2.jar" basedir="${build.dir}">
55         <manifest>
56             <attribute name="Main-Class" value="work5.ActionDemo"/>
57         </manifest>
58     </jar>
59 </target>
60
61 <!-- Запуск демо-програми -->
62 <target name="run" depends="compile,javadoc" description="Run demo program">
63     <java classname="work5.ActionDemo" fork="true">
64         <classpath>
65             <pathelement path="${build.dir}"/>
66         </classpath>
67     </java>

```

```

67     </target>
68
69     <!-- Очищення результатів збірки -->
70     <target name="clean" description="Delete build, dist and doc directories">
71         <delete dir="${build.dir}"/>
72         <delete dir="${dist.dir}"/>
73         <delete dir="${doc.dir}"/>
74     </target>
75
76 </project>

```

Перевірка:

```

1 PS C:\Users\artem\Desktop\repos\university_roadmap\semestr3\software
   ↪ engineering\labs\lab5\task2> ant -p
2 Buildfile: C:\Users\artem\Desktop\repos\university_roadmap\semestr3\software
   ↪ engineering\labs\lab5\task2\build.xml
3
4 Main targets:
5
6 clean    Delete build, dist and doc directories
7 compile  Compile Java sources
8 init     Create build, dist and doc directories
9 jar      Create runnable JAR archive
10 javadoc  Generate Javadoc documentation
11 run      Run demo program
12 Default target: run

```

Вихідний код буде на віддаленому репозиторії, посилання на яке буде додане разом із звітом.

Загальна структура проекту після виконання збірки:

```

1 PS C:\Users\artem\Desktop\repos\university_roadmap\semestr3\software
   ↪ engineering\labs\lab5\task2> tree /f /a
2 Folder PATH listing
3 Volume serial number is 1A5B-F8AF
4 C:..
5 |   build.xml
6 |
7 +---build
8 |   \---work5
9 |       Action.class
10 |       ActionDemo.class
11 |       CopyAction.class
12 |       Invoker.class
13 |       MacroAction.class
14 |       OpenFileAction.class
15 |       PasteAction.class
16 |       SaveFileAction.class
17 |
18 +---dist
19 +---doc
20 |   |   allclasses-index.html

```

```

21 | | allpackages-index.html
22 | | copy.svg
23 | | element-list
24 | | help-doc.html
25 | | index-all.html
26 | | index.html
27 | | link.svg
28 | | member-search-index.js
29 | | module-search-index.js
30 | | overview-tree.html
31 | | package-search-index.js
32 | | script.js
33 | | search-page.js
34 | | search.html
35 | | search.js
36 | | stylesheet.css
37 | | tag-search-index.js
38 | | type-search-index.js
39 | |
40 | +---legal
41 | |     COPYRIGHT
42 | |     jquery.md
43 | |     jqueryUI.md
44 | |     LICENSE
45 | |
46 | +---resources
47 | |     glass.png
48 | |     x.png
49 | |
50 | +---script-dir
51 | |     jquery-3.7.1.min.js
52 | |     jquery-ui.min.css
53 | |     jquery-ui.min.js
54 | |
55 | \---work5
56 | |     Action.html
57 | |     ActionDemo.html
58 | |     CopyAction.html
59 | |     Invoker.html
60 | |     MacroAction.html
61 | |     OpenFileAction.html
62 | |     package-summary.html
63 | |     package-tree.html
64 | |     package-use.html
65 | |     PasteAction.html
66 | |     SaveFileAction.html
67 | |
68 | \---class-use
69 | |     Action.html
70 | |     ActionDemo.html
71 | |     CopyAction.html
72 | |     Invoker.html
73 | |     MacroAction.html
74 | |     OpenFileAction.html

```

```

75 |             PasteAction.html
76 |             SaveFileAction.html
77 |
78 | \---src
79 |     \---work5
80 |         Action.java
81 |         ActionDemo.java
82 |         CopyAction.java
83 |         Invoker.java
84 |         MacroAction.java
85 |         OpenFileAction.java
86 |         PasteAction.java
87 |         SaveFileAction.java

```

Скріншот наявності документації:

Package work5

package work5

All Classes and Interfaces	Interfaces	Classes
Class	Description	
Action	Інтерфейс, що представляє абстрактну дію користувача.	
ActionDemo	Демонстраційний клас для показу роботи шаблону.	
CopyAction	Конкретна дія, що моделює операцію копіювання.	
Invoker	Клас-ініціатор, який відповідає за "натискання" кнопок.	
MacroAction	Макро-дія, що представляє собою послідовність інших дій.	
OpenFileAction	Конкретна дія, що моделює відкриття файлу.	
PasteAction	Конкретна дія, що моделює операцію вставки.	
SaveFileAction	Конкретна дія, що моделює збереження файлу.	

Рис. 3: Скріншот документації Javadoc для 2 завдання

UML-діаграма проекту:

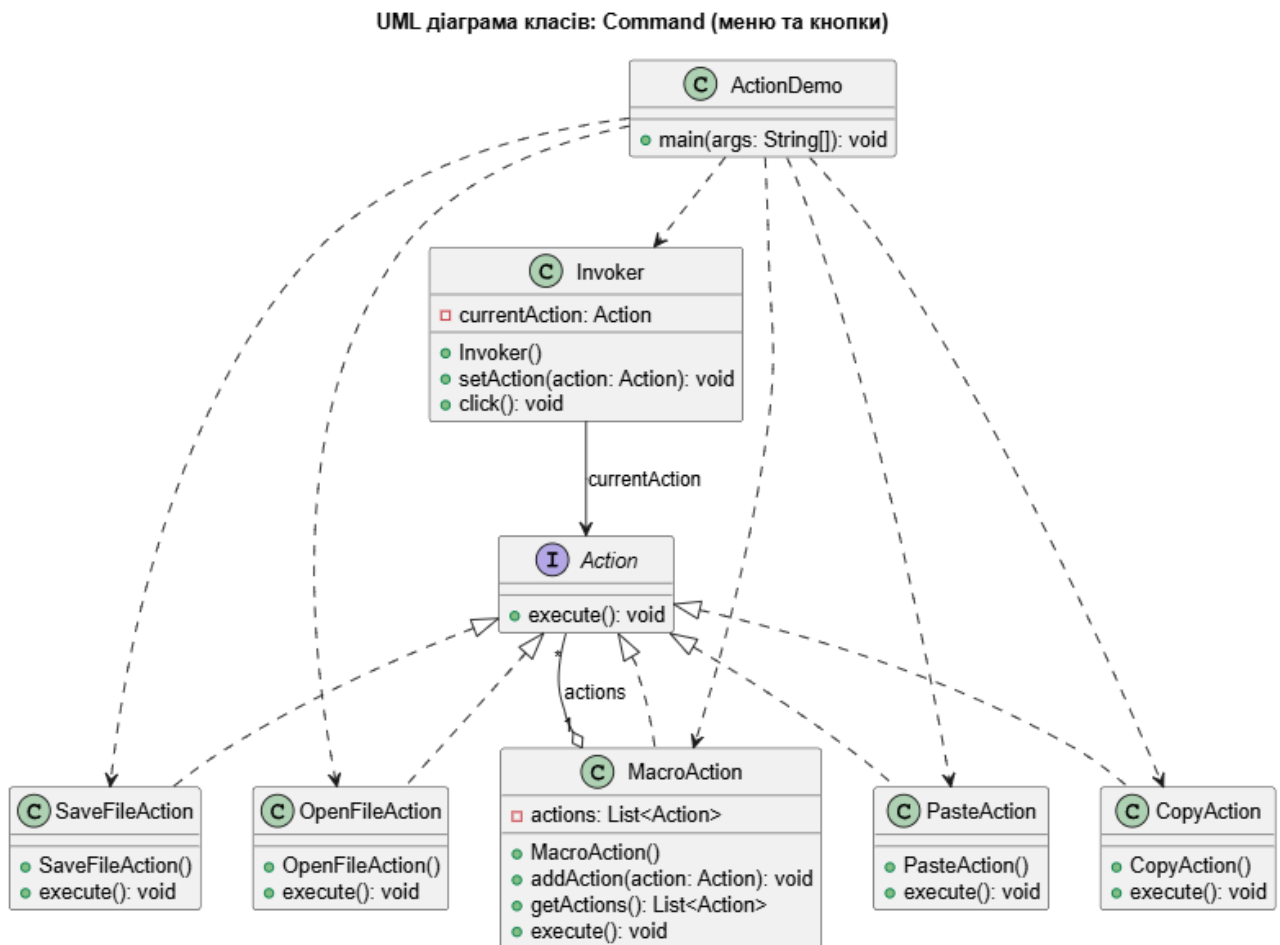


Рис. 4: UML-діаграма проекту для 2 завдання

У пакеті `work5` реалізовано модель взаємодії користувача з пунктами меню та кнопками інструментальної панелі відповідно до варіанту. Базовий інтерфейс `Action` визначає єдиний метод `execute()`, який описує абстрактну дію. Конкретні класи `SaveFileAction`, `OpenFileAction`, `CopyAction` та `PasteAction` реалізують інтерфейс `Action` і моделюють окремі команди (збереження, відкриття, копіювання, вставку).

Клас `MacroAction` також реалізує `Action` і містить колекцію об'єктів `Action`; при виклику `execute()` він послідовно виконує усі вкладені дії, що відповідає вимозі формування макро-реакцій. Клас `Invoker` зберігає посилання на поточну дію `Action` та в методі `click()` просто викликає `execute()` цієї дії, тобто динамічна зміна поведінки досягається через `setAction()` без використання умовних операторів `if/switch`. У класі `ActionDemo` в методі `main()` створюються окремі дії, макро-дія та об'єкт `Invoker`, демонструється виконання одиночних команд і макро-команди, що підтверджує коректну реалізацію шаблону *Command* та повну відповідність умовам завдання.

Висновок: У ході виконання лабораторної роботи було закріплено теоретичні знання щодо шаблонів поведінки у проектуванні ПЗ та продемонстровано їх практичне застосування на прикладах.

У першому завданні було реалізовано модель мережевої структури з використанням шаблону *Visitor* (та фактично *Composite* для ієрархії елементів). Інтерфейс `NetworkElement` і конкретні класи `Cable`, `Server`, `Workstation`, `Network` дозволили побудувати деревоподібну структуру мережі, а відвідувачі `DisplayVisitor` та `CostCalculatorVisitor` показали можливість додавання нових операцій над елементами без зміни їхнього коду. Це відповідає принципам розширюваності та слабкого зв'язку між структурами даних та операціями над ними.

У другому завданні було реалізовано шаблон *Command* для моделювання взаємодії користувача з меню та кнопками. Інтерфейс `Action` та конкретні команди `SaveFileAction`, `OpenFileAction`, `CopyAction`, `PasteAction` інкапсулюють окремі дії, тоді як клас `Invoker` делегує виконання поточній команді без використання умовних конструкцій.

Клас `MacroAction` продемонстрував можливість об'єднання кількох команд у єдину макро-операцію. Таким чином, було показано, як шаблон *Command* забезпечує гнучку зміну поведінки об'єкта під час виконання програми.

Окрім цього, для обох завдань було налаштовано збірку проєктів за допомогою системи `Ant` та автоматичну генерацію документації засобами `Javadoc`, що сприяє підвищенню якості супроводу коду. У підсумку лабораторна робота дозволила не лише засвоїти призначення та структуру шаблонів *Visitor* і *Command*, але й отримати практичні навички їх реалізації в мові Java в контексті реалістичних задач.